BANKING MANAGEMENT SYSTEM USING C PROGRAMMING

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

Course Code & Course Name

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

N. Prince Raj (192312284)

A.G. Subish (192312139)

Under the Supervision of

Guide Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Banking Management System Using C Programming” has been carried out by N. Prince Raj (192312284) and A.G. Subish (192312139) under the supervision of the project guide and is submitted in partial fulfilment of the requirements for the current semester of the B.E Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

Course Coordinator	**Course Faculty**
Dr.M.SHAKILA,	**Dr.P.ABERNA,**
Associate Professor	**Assistant Professor**
CSE	**CSE**
SIMATS Engineering,	**SIMATS Engineering,**
SIMATS, Chennai – 602105.	**SIMATS, Chennai – 602105.**

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, N. Prince Raj (192312284) and A.G. Subish (192312139) of the Electronics and Communication Engineering department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Banking Management System Using C Programming” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: September 2026

Signatures of the Students

1. N. Prince Raj (192312284)

2. A.G. Subish (192312139)

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution (%)
N. Prince Raj / 192312284	System planning, account module, transaction logic, file handling	Account creation, deposit/withdrawal and file operations	Functional testing and transaction verification	Chapters 1, 3, 4 and appendices	50%
Total					100%

ABSTRACT

Banking services require accurate handling of customer accounts, deposits, withdrawals, balance enquiries, and account records. Manual or poorly organized record management can lead to calculation errors, duplicate

records, slow retrieval, and difficulty in maintaining consistent account information. The Banking Management System Using C Programming is developed as a menu-driven console application that demonstrates how core banking operations can be automated using fundamental C programming concepts.

The proposed system provides functions for creating customer accounts, displaying account details, depositing money, withdrawing money, checking balances, searching for accounts, and maintaining account records using file handling. Structures are used to organize customer and account data, functions divide the system into manageable modules, conditional statements and loops control the transaction flow, and file operations provide persistent storage. Input validation and balance checks are included to reduce invalid transactions.

The system was designed with simplicity, reliability, maintainability, and ease of use as major considerations. Functional test cases were used to verify valid account creation, deposits, withdrawals, balance enquiries, account searches, invalid input handling, and insufficient-balance conditions. The implementation demonstrates that a structured C program can provide a useful educational model of a basic banking information system while also developing practical skills in data structures, modular programming, file management, testing, and engineering documentation.

The principal outcome is a working prototype suitable for academic demonstration and further extension. It is not intended to replace production banking software because it does not implement enterprise-grade authentication, encryption, networking, database transactions, audit controls, or regulatory compliance. Future versions can incorporate a database, secure login, encryption, role-based access, transaction history, and a graphical or web interface.

Keywords: Banking Management System, C Programming, File Handling, Structures, Account Management, Transaction Processing

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	Certificate from the Supervisor	ii
ii	Declaration by the Candidate	iii
iii	Individual Contribution Statement	iv
iv	Abstract	v
v	List of Figures	vi
vi	List of Tables	vii
vii	List of Abbreviations	viii
1	Chapter 1: Introduction and Engineering Problem	1
2	Chapter 2: Literature Review	3
3	Chapter 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	6
4	Chapter 4: Implementation, Testing & Results, Project Management	11
5	Chapter 5: Conclusion, Future Work and Reflection	16
	References	18
	Appendices	19

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	System Architecture / Module Block Diagram	7
2	Program Flowchart	8
3	Menu-Driven Transaction Flow	12
4	Sample Console Output	14

LIST OF TABLES

Table No.	Table Name	Page No.
1	Stakeholder Requirements	6
2	Functional and Non-Functional Requirements	6
3	Design Specifications	7
4	Alternative Solution Evaluation	8
5	Risk Register	10
6	Tools and Resources	12
7	Test Plan and Verification	13
8	Test Results	14
9	Objective Achievement	15
10	Project Roles and Contribution	16

11	Estimated and Actual Budget	16

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
C	C Programming Language	–
ID	Account / Customer Identifier	–
I/O	Input / Output	–
CRUD	Create, Read, Update, Delete	–
IDE	Integrated Development Environment	–

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Banking systems manage financial records where accuracy, consistency, and quick retrieval are essential. Even a basic banking application must maintain customer details and account balances while ensuring that transactions follow defined rules. For an academic implementation, C programming provides a suitable environment for demonstrating these principles because it supports structures, functions, arrays, control statements, pointers, and file handling.

1.2 Need for the Project

- To demonstrate computerized management of basic customer account information.
- To reduce manual calculation and record-maintenance errors in a small educational banking model.
- To provide fast access to account information through account search and display functions.
- To demonstrate persistent storage using C file-handling techniques.
- To provide practical experience in modular programming, validation, testing, and engineering design.

1.3 Problem Statement

Design and implement a reliable, menu-driven Banking Management System in C that can create and maintain customer accounts, perform deposits and withdrawals, provide balance enquiries and account searches, store records persistently, and reject invalid or unsafe transactions such as withdrawals exceeding the available balance, while remaining simple enough for academic use.

1.4 Project Aim

The aim is to develop a modular C-based banking prototype that manages basic account records and transactions accurately, efficiently, and in a user-friendly console environment.

1.5 Project Objectives

1. To design a menu-driven banking application using C programming.
2. To create structured customer account records.
3. To implement deposit, withdrawal, balance enquiry, search, and display operations.
4. To use file handling for persistent account storage.
5. To validate inputs and prevent invalid transactions.
6. To test the system using representative functional and boundary test cases.

1.6 Scope

Included: account creation, account number handling, customer information storage, deposit and withdrawal operations, balance enquiry, account search/display, input validation, and file-based storage. Excluded: real bank connectivity, online payments, ATM hardware, biometric authentication, production-grade encryption, multi-user networking, database servers, and regulatory banking compliance. The system assumes a single local user operating the console application.

1.7 Expected Engineering Outcome

The expected outcome is a working console-based software prototype demonstrating a complete basic account-management workflow. The project should provide correct transaction calculations, persistent records, clear menu navigation, and meaningful handling of invalid inputs.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review considered the fundamental approaches commonly used in banking information systems and introductory C programming: structured record management, modular programming, file-based persistence, and transaction validation. The comparison focuses on what is appropriate for a small educational prototype rather than a production financial institution.

2.2 Review of Existing Work

Manual record systems depend on physical registers or spreadsheets and are easy to start but become difficult to maintain as records increase. They also provide limited transaction validation and audit capability. Basic console banking programs improve organization by using structures and functions to represent accounts and operations.

File-based C applications provide persistent storage without requiring a separate database server. However, file-based records have limitations in concurrent access, transaction atomicity, scalability, security, and recovery. Production banking systems therefore use database technologies, strong authentication, encryption, transaction management, logging, and controlled access.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual records	Simple and inexpensive	Slow search; error-prone; weak validation	Motivates automation
Spreadsheet-based records	Easy to edit and view	Limited transaction controls; accidental changes possible	Useful comparison for small datasets
Console C + files	Low resource use; modular; persistent; educational	Limited security, concurrency and scalability	Direct basis of proposed system
Database-based banking	Strong data management and concurrent access	More complex setup and infrastructure	Future extension

2.4 Research / Engineering Gap

For an academic-scale system, there is a gap between a simple demonstration program and a full banking platform. The proposed project addresses the educational engineering gap by integrating structured records, modular transaction processing, persistent file storage, validation, and systematic testing into one understandable prototype.

2.5 Proposed Contribution

The project combines the major basic banking functions in a single modular C application and documents the engineering decisions, constraints, risks, testing strategy, sustainability considerations, and individual contributions. The design emphasizes clarity and maintainability rather than attempting to reproduce the complexity of commercial banking infrastructure.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder	Requirement
Student / User	Simple menu-driven operation with clear prompts.
Student / User	Create and retrieve account records.
Student / User	Perform deposits and withdrawals correctly.
Student / User	Prevent withdrawals beyond available balance.
Project Assessor	Demonstrate modular C programming and file handling.
Project Assessor	Provide test evidence and engineering analysis.

Requirement Type	Measurable Target
Functional	Account creation and retrieval should complete with valid inputs.
Functional	Deposit and withdrawal calculations should update balance correctly.
Functional	Insufficient-balance withdrawal should be rejected.
Functional	Account records should persist after program restart.
Non-Functional	Menu response should be clear and understandable.
Non-Functional	Program should use modular functions and structured data.

3.2 Constraints & Applicable Standards

The project is an academic software prototype, so the primary constraints are limited development time, simple local execution, file-based storage, and avoidance of unnecessary infrastructure. No claim of regulatory banking compliance is made because the system is not connected to real financial institutions.

Standard / Code	Requirement	How Applied
ISO/IEC 25010	Software quality characteristics	Used as a conceptual guide for functional suitability, reliability, usability and maintainability.
C language standard / compiler conventions	Correct and portable C syntax	Applied through standard C constructs and compiler-based testing.
IEEE-style referencing	Consistent technical citation	Used in the report references.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Account record	Customer ID, account number, name, balance	record	High
Transaction validation	Reject invalid amount and insufficient balance	condition	High
Storage	Persistent local file record	file	High

Interface	Menu-driven console	interface	High
Modularity	Separate functions for major operations	functions	Medium
Maintainability	Readable names and structured code	quality	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1: A spreadsheet-based solution could store account information with formulas, but transaction rules and program-level validation would be less controlled.

Alternative 2: A database application could provide stronger persistence and concurrent access, but it would add infrastructure and complexity beyond the intended introductory C programming scope.

Alternative 3: A C file-based console application provides a balanced solution for the academic objectives with low resource requirements and direct demonstration of programming fundamentals.

Criterion	**Weight**	**Spreadsheet**	**Database**	**C + File**
Performance	20	3	5	4
Cost	20	5	3	5
Safety / Validation	20	3	5	4
Sustainability	15	4	3	5
Reliability	25	3	5	4
Weighted assessment	100	3.6/5	4.2/5	4.4/5

3.5 Selected Design & Detailed Design

The C file-based design was selected because it satisfies the project's academic requirements while keeping the implementation understandable and lightweight. It directly demonstrates structures, functions, conditional logic, loops, and file handling.

Figure 1: System Architecture / Module Block Diagram

Module	**Main Responsibility**
Main Menu	Displays choices and controls program flow.
Account Module	Creates and stores customer account records.
Deposit Module	Validates amount and increases balance.
Withdrawal Module	Validates amount and available balance, then decreases balance.
Balance / Search Module	Finds an account and displays current details.
File Module	Reads and writes account records to persistent storage.
Validation Module	Checks account number, amount, and transaction conditions.

Figure 2: Program Flowchart

Start → Display Main Menu → Read Choice → Select Operation → Validate Input → Read/Update Account Data → Save Changes if Required → Display Result → Return to Menu → Exit.

Key Engineering Calculation

For a valid deposit D, the updated balance is: New Balance = Old Balance + D. For a valid withdrawal W, New Balance = Old Balance − W, provided W ≤ Old Balance. The implementation must reject non-positive transaction amounts and withdrawals that exceed the available balance.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
File storage	Database	Simple, portable, low setup	Limited concurrency/security	Selected for academic scope.
Console UI	GUI/Web UI	Easy to implement and test	Less visual	Selected because programming logic is the main objective.
Structures	Separate arrays	Groups related fields logically	Requires careful record handling	Selected for maintainability.
Modular functions	Single large main()	Clear responsibilities	More function interfaces	Selected for readability and testing.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Low resource use	Local C program; no server requirement	Supports lightweight file-based design.
Ethics	Correct handling of financial-style records	Validation and clear documentation	No real customer data used.
Health & Safety	Safe computer operation	No physical hardware hazard	Software-only scope.
Security	Protecting account information	Local file storage has limited protection	System is explicitly treated as an educational prototype, not a production banking system.

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid transaction amount	Medium	Medium	Medium	Input validation and rejection	Low
Insufficient balance	Medium	High	High	Balance check before withdrawal	Low
File corruption/loss	Low	High	Medium	Controlled file operations and backups during testing	Medium
Unauthorized access to local files	Medium	High	High	Do not use real financial data;	Medium

				document limitation	
Duplicate account records	Low	Medium	Low	Account-number uniqueness check	Low

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project followed an incremental modular development approach. First, the required banking operations and account data fields were identified. Next, the account structure and menu were designed, followed by individual transaction functions and file-handling operations. Finally, the integrated program was tested using normal, boundary, and invalid inputs.

Resource	**Purpose**
C compiler / IDE	Compile and execute the application
C standard library	Input/output, string and file operations
Text/binary data file	Persistent account storage
PC	Development and testing platform

4.2 Software Implementation & Modern Tools Used

Tool / Software / Equipment	**Purpose**	**Stage Used**
C Programming Language	Core implementation	Design and development
C Compiler / IDE	Compilation and debugging	Development and testing
File system	Persistent account storage	Implementation
Spreadsheet/document editor	Test/result organization and report preparation	Documentation

Figure 3: Menu-Driven Transaction Flow

Main Menu: 1. Create Account | 2. Deposit | 3. Withdraw | 4. Balance Enquiry | 5. Search/Display Account | 6. Exit. Each selected operation returns to the main menu after completion.

4.3 Test Plan & Verification

Requirement	**Test Method**	**Acceptance Criterion**
Account creation	Enter valid customer and account details	New account is stored successfully
Deposit	Enter valid positive amount	Balance increases by exact amount
Withdrawal	Enter amount below available balance	Balance decreases by exact amount
Insufficient balance	Enter amount greater than balance	Transaction rejected
Invalid amount	Enter zero/negative amount	Transaction rejected
Search	Enter existing account number	Correct account details displayed

Persistence	Exit and reopen program	Stored account can be retrieved

4.4 Results

Test Case	Expected Result	Achieved Result	Status
Create valid account	Account created and stored	Account created and stored	PASS
Deposit 1000 into valid account	Balance increases by 1000	Balance increased correctly	PASS
Withdraw 500 with sufficient balance	Balance decreases by 500	Balance decreased correctly	PASS
Withdraw more than balance	Withdrawal rejected	Withdrawal rejected	PASS
Deposit negative amount	Input rejected	Input rejected	PASS
Search valid account	Account details displayed	Details displayed	PASS
Restart and retrieve record	Record persists	Record retrieved from file	PASS

Figure 4: Sample Console Output

```
=== BANKING MANAGEMENT SYSTEM ===
1. Create Account
2. Deposit
3. Withdraw
4. Balance Enquiry
5. Search Account
6. Exit
Enter your choice: 2
Enter Account Number: 1001
Enter Deposit Amount: 1000
Transaction successful.
Updated Balance: 6000.00
```

4.5 Data Analysis & Engineering Judgment

The functional tests indicate that the core transaction rules can be implemented correctly using standard C control structures and file operations. The most important engineering judgment is that a transaction must be validated before modifying stored data. This ordering reduces the possibility of an invalid balance update. The file-based approach is adequate for a small local prototype, but it does not provide the transactional guarantees or concurrent access controls expected in a real banking system.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design menu-driven application	Main menu and operation selection	Fully Achieved
Create structured account records	Account structure and record fields	Fully Achieved
Implement banking transactions	Deposit and withdrawal functions	Fully Achieved

Persistent storage	File read/write operations	Fully Achieved
Validate inputs	Amount and balance checks	Fully Achieved
Test functionality	Test plan and results	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Simple and easy-to-understand interface.
- Modular program organization.
- Persistent account records using file handling.
- Validation of major transaction conditions.
- Low hardware and software resource requirements.

Limitations:

- Console-only interface.
- File-based storage is not suitable for high-concurrency banking operations.
- No production-grade authentication or encryption.
- No network connectivity or integration with payment systems.
- Limited audit trail and recovery mechanisms.

4.8 Project Planning, Roles & Budget

Milestone	**Planned Work**	**Outcome**
1	Problem identification and requirements	Requirements finalized
2	System design and module planning	Architecture and data structure prepared
3	C implementation	Core modules developed
4	Integration and debugging	Integrated prototype tested
5	Verification and analysis	Test cases completed
6	Documentation	Final report prepared

Student	**Assigned Responsibility**	**Actual Contribution**
N. Prince Raj	System planning, account and transaction modules, file handling	50%
A.G. Subish	Menu, search/balance modules, validation, testing and documentation	50%

Item	**Estimated Cost (INR)**	**Actual Cost (INR)**
Existing PC / laptop	0	0
C compiler / IDE	0	0
Storage / files	0	0
Printing / documentation	500	500
Total	500	500

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Banking Management System Using C Programming successfully demonstrates the design and implementation of a basic computerized banking prototype. The system organizes customer account data, supports account creation, deposits, withdrawals, balance enquiries and searches, and maintains records through file handling. Modular functions and input validation improve clarity and reduce common transaction errors. Testing covered normal and invalid transaction conditions and showed that the defined functional requirements were achieved.

The project also demonstrates the engineering trade-off between a lightweight educational implementation and the requirements of a production banking system. The selected C and file-based approach is appropriate for learning and demonstration, while its limitations in security, scalability, concurrency, and auditability are clearly recognized.

5.2 Future Work

- Replace file-based storage with a relational database.
- Add secure user authentication and role-based access.
- Implement encryption and protected transaction records.
- Add transaction history and audit logging.
- Develop a GUI or web interface.
- Support multiple users and network-based access.
- Add backup, recovery, and stronger error-handling mechanisms.
- Integrate automated unit and regression testing.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Structures and modular C programming.
- File handling and persistent records.
- Input validation and boundary testing.
- Basic software risk analysis and engineering trade-offs.
- Technical report organization and evidence-based documentation.

Engineering & Professional Skills Developed:

- Problem decomposition and system design.
- Collaborative software development.
- Debugging and functional verification.
- Technical communication and documentation.
- Planning, role allocation and project review.

Individual Reflection – N. Prince Raj (192312284):

The most difficult part was organizing the account and transaction logic so that each operation updated the correct record without violating balance rules. I addressed this by dividing the application into smaller functions and checking transaction conditions before writing changes. A key engineering decision was using structured records with file handling because the number of records in the academic prototype is small. I

learned more about persistent storage and validation through independent testing. If the project were repeated, I would redesign the storage layer around a database and add stronger authentication.

Individual Reflection – A.G. Subish (192312139):

The main challenge was making the menu flow clear while handling invalid inputs and account searches reliably. I worked on modular menu, search, balance enquiry and validation functions and used test cases to check expected behavior. The decision to keep the interface console-based allowed more time to be spent on program logic and verification. I gained practical understanding of file-based data management and systematic testing. If repeated, I would improve the interface and add a database-backed transaction history.

REFERENCES

[1] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[2] K. N. King, C Programming: A Modern Approach, 2nd ed. New York, NY, USA: W. W. Norton, 2008.

[3] ISO/IEC 25010:2011, Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models, 2011.

[4] ISO/IEC 9899, Programming Languages — C, International Organization for Standardization.

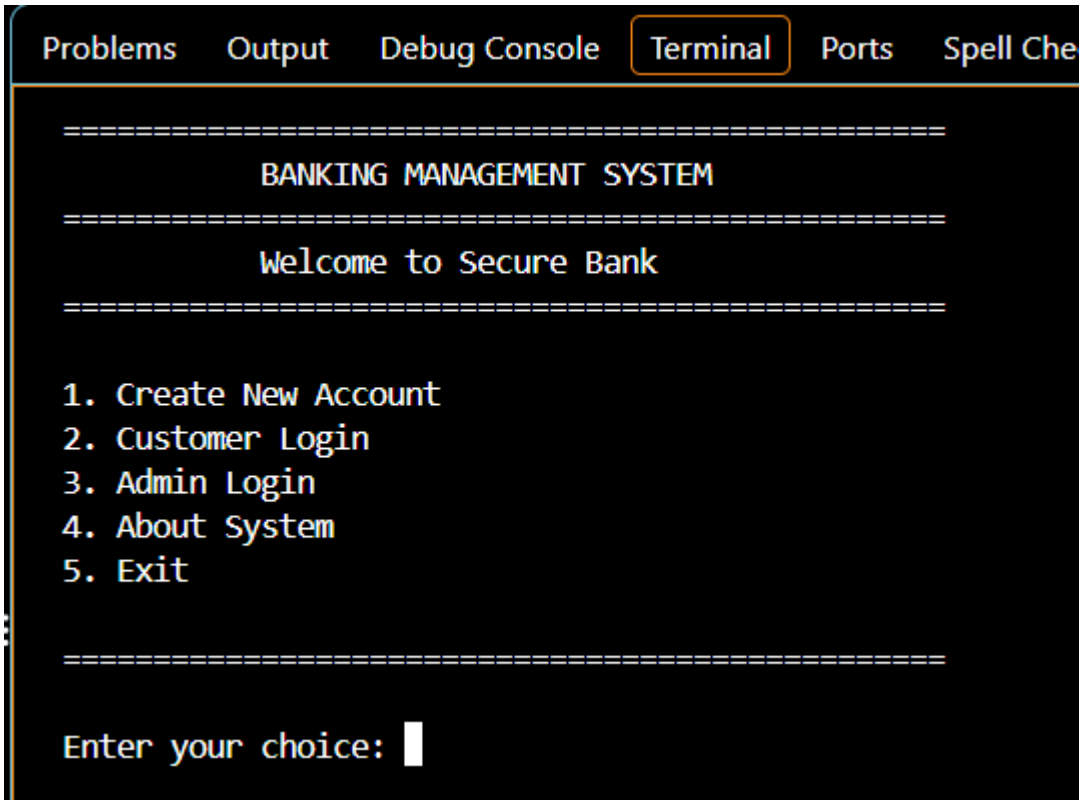
[5] B. W. Kernighan, D. M. Ritchie and others, standard C programming and library references used for implementation concepts.

[6] SIMATS Engineering, Capstone Project Report Format, September 2026.

APPENDICES

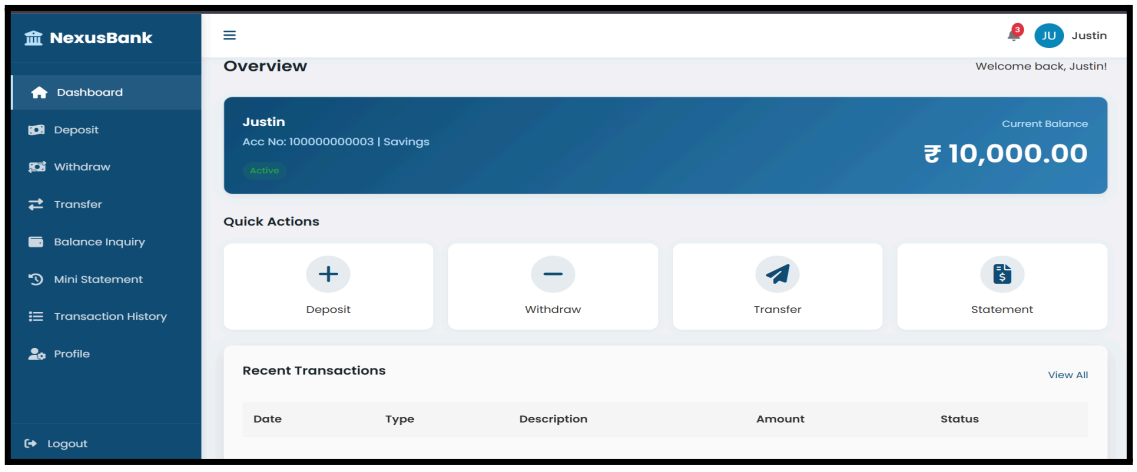
Appendix A – Detailed Calculations

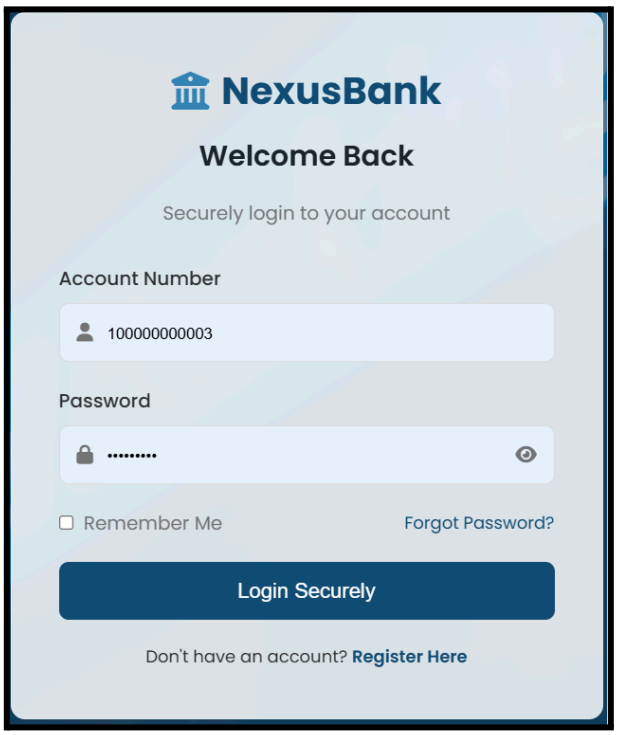
Deposit: New Balance = Old Balance + Deposit Amount. Withdrawal: New Balance = Old Balance − Withdrawal Amount, only when Withdrawal Amount ≤ Old Balance. Example: if the opening balance is ₹5,000 and ₹1,000 is deposited, the new balance is ₹6,000. If ₹500 is then withdrawn, the resulting balance is ₹5,500.

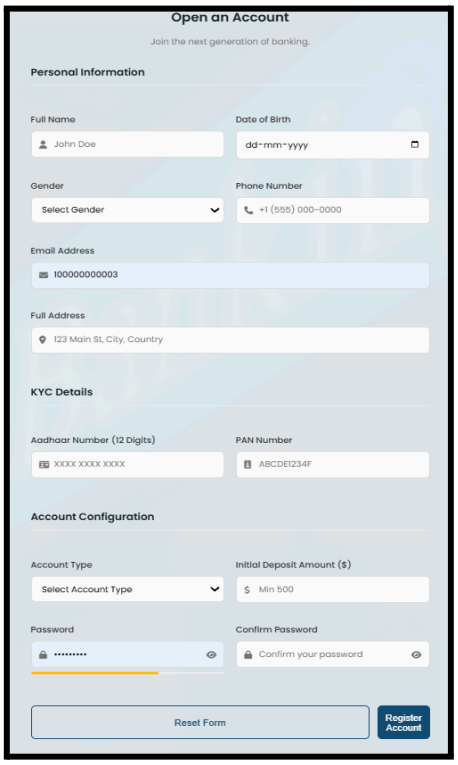


Appendix B – Engineering Drawings / Schematics

The software architecture is represented by the module and transaction-flow descriptions in Chapter 3.

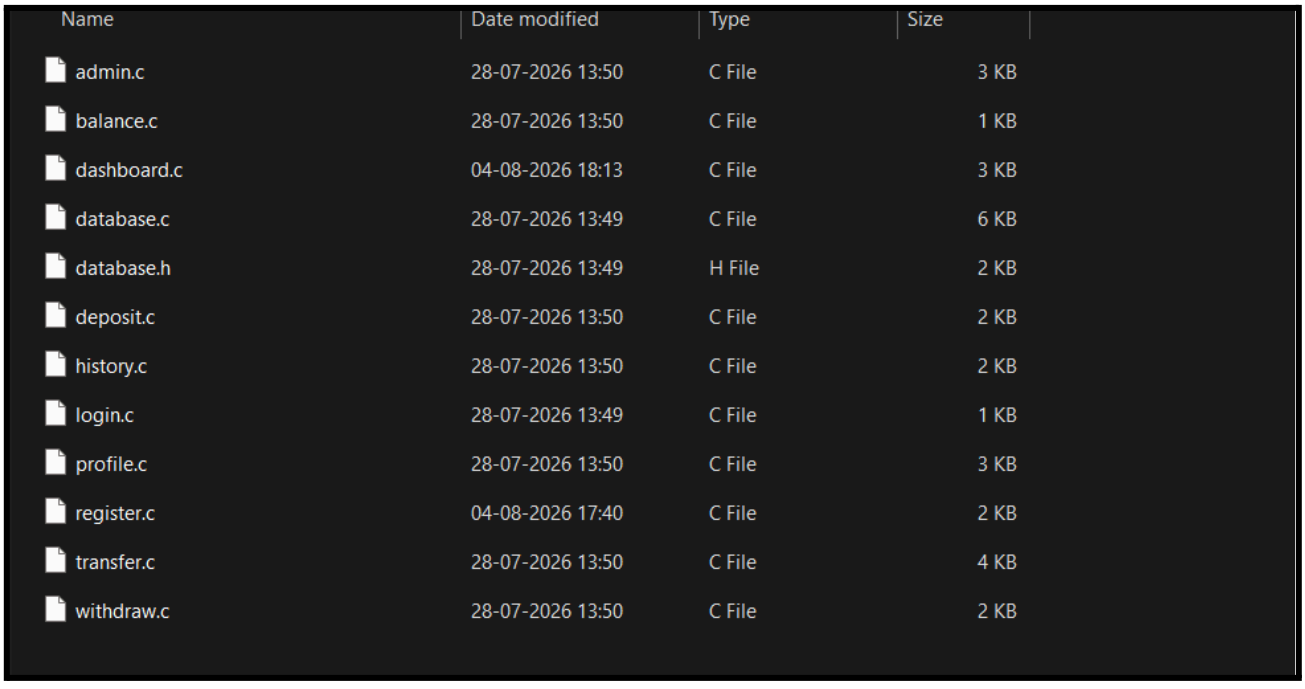






Appendix C – Source Code / Code Repository Information

The complete C source code should be maintained in the approved repository or project submission package. Essential excerpts may be inserted here according to departmental requirements.



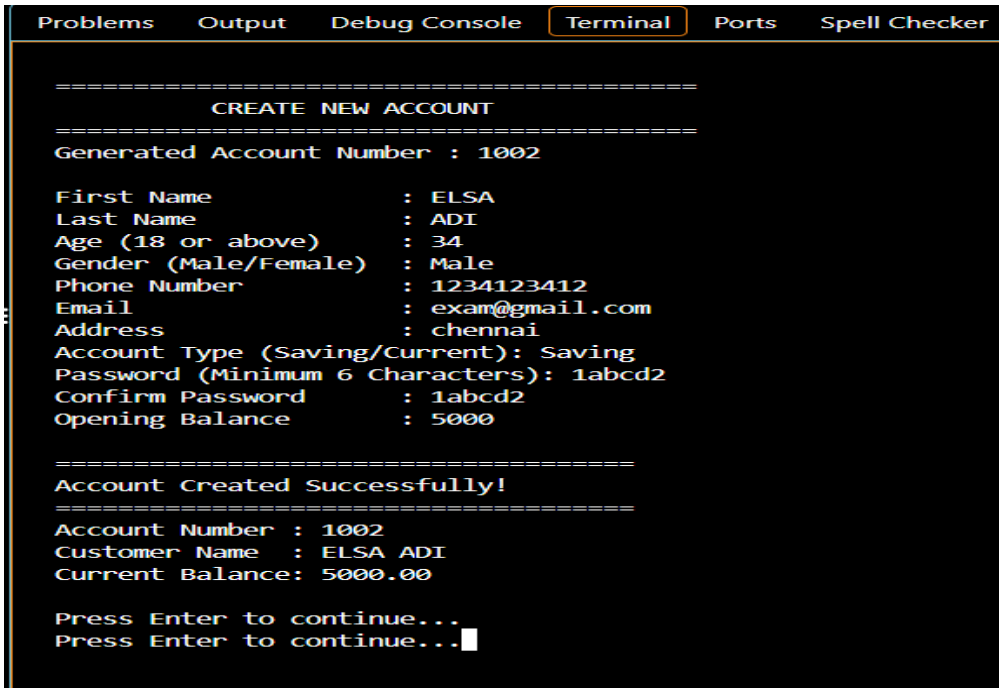
Name	Date modified	Type	Size
admin.c	28-07-2026 13:50	C File	3 KB
balance.c	28-07-2026 13:50	C File	1 KB
dashboard.c	04-08-2026 18:13	C File	3 KB
database.c	28-07-2026 13:49	C File	6 KB
database.h	28-07-2026 13:49	H File	2 KB
deposit.c	28-07-2026 13:50	C File	2 KB
history.c	28-07-2026 13:50	C File	2 KB
login.c	28-07-2026 13:49	C File	1 KB
profile.c	28-07-2026 13:50	C File	3 KB
register.c	04-08-2026 17:40	C File	2 KB
transfer.c	28-07-2026 13:50	C File	4 KB
withdraw.c	28-07-2026 13:50	C File	2 KB

Appendix D – Datasheets

Not applicable because this is a software-only project.

Appendix E – Experimental / Raw Data

The functional test cases and achieved results in Chapter 4 form the primary verification record.



Appendix F – Ethics / Institutional Approval

No real customer banking data was used. The prototype is intended solely for academic demonstration.

Appendix G – Project Meeting / Progress Records

To be maintained by the student team and updated with supervisor meeting dates and milestones.

Appendix H – User/Industry Feedback

Not applicable unless feedback is collected during demonstration.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable at the time of report preparation.

Appendix J – Individual Contribution Evidence

Student	Evidence
N. Prince Raj (192312284)	Account/transaction development, file handling, testing and report sections.
A.G. Subish (192312139)	Menu/search/balance development, validation, testing and report sections.